DAT32-3

# Machine Learning I

Linear and logistic models: classification, its evaluation toolkit, and the judgment to keep a model simple

---

Classification is regression's discipline applied to a different question — and the hard part is no longer the model, it is the evaluation.

Albert School — 22 hours

# Contents



# Preface

In **Intro to Machine Learning** (DAT32-1) you walked the complete machine-learning cycle once, on linear regression: you defined a target, split the data, beat a baseline, fitted with ordinary least squares and with gradient descent, diagnosed overfitting from learning curves, controlled it with L1 and L2 regularization, estimated performance honestly with k-fold cross-validation, tuned hyperparameters on a validation set, extended to multiple features, and packaged everything in a reproducible scikit-learn pipeline. That cycle is the spine of this course too. We do not re-teach it; we assume you own it.

What changes here is the **question**. Regression predicts a number — a rent, a demand, a revenue. But most decisions a business actually makes are not "how much?" they are "which one?" and "yes or no?": will this customer churn, is this transaction fraud, should this email reach the inbox, will this loan default. That is **classification**, and it is the task type that dominates applied machine learning.

The model we anchor on is **logistic regression**. The name is a small cruelty — it is a classification model, not a regression one — but the name is also a promise: it is linear regression's twin. It is a weighted sum of features, exactly as before; we simply pass that sum through one function that turns it into a probability. Everything you know about coefficients, gradient descent and regularization transfers almost unchanged.

The genuinely new material is **evaluation**. In regression, "how wrong?" had a clean answer in the units of the target. In classification a single error rate hides life-or-death distinctions: a cancer screen that misses a tumour and one that raises a false alarm are both "wrong," but they are not equally wrong, and no business treats them as such. The bulk of this book is therefore the classification evaluation toolkit — the confusion matrix, precision and recall, the F1 score, the ROC curve and AUC, and the decision threshold — because choosing and defending the right metric is where classification is won or lost.

We close where the course's judgment lives: feature scaling, feature engineering for linear models, regularization for logistic regression, and the case for keeping a model simple. A running example carries the whole book — **predicting which subscribers to a streaming service will cancel next month** (customer churn) — so that every new tool refers to a decision you can picture.

# 1 From regression to classification

Your streaming company wants to know, for each subscriber, whether they will cancel ("churn") in the coming month, so the retention team can call the ones most at risk. You have last year's data: for every subscriber, their monthly watch hours, tenure, support tickets, plan price — and whether they ultimately churned. This looks like a regression problem you could solve with DAT32-1′s tools. It is not, and seeing exactly why motivates everything that follows.

> **Definition 1** **Classification** is supervised learning where the target is a **discrete category** rather than a continuous number. **Binary classification** has two categories (churn / stay, fraud / legitimate); **multi-class classification** has more than two (which of five plans a customer will pick).

Three differences force new tools.

**The output is discrete.** The label is "churn" or "stay," not a number on a continuous scale. If you ran ordinary linear regression on a 0/1 target, the model would happily predict 1.4 or $-0.3$ — values that mean nothing as a category and cannot be read as anything else either.

**We want a probability, not just a verdict.** The retention team has budget to call 200 people, not 50 000. They do not want a flat "will churn / won't churn"; they want each subscriber ranked by **how likely** they are to churn, so they can spend the budget on the riskiest. A good classifier outputs a probability between 0 and 1, and the verdict is a second step.

> **Worked example — why a probability beats a verdict.** Two subscribers are both flagged "will churn." One is at 0.96 probability, the other at 0.51. With 200 calls to make and 4 000 flags, you cannot treat them alike — you call the 0.96 first. A model that only emits "yes/no" has thrown away the information you most need. This is why classification models are built to emit probabilities and the yes/no is recovered afterward by a **threshold**.

**Errors are asymmetric.** In rent prediction, being €50 too high and €50 too low were roughly equally bad. In classification the two ways of being wrong usually cost wildly different amounts. Failing to flag a churner who then leaves costs you a customer's lifetime value, perhaps €600. Flagging a loyal customer who was never going to leave costs you one retention call, perhaps €4. Treating these as "one error each" is a business mistake, and the evaluation toolkit exists precisely to respect the difference.

**Common pitfall** Do not fit ordinary linear regression to a 0/1 label and threshold its output at 0.5. It "works" often enough to be tempting, but its predictions are unbounded (values above 1 and below 0 are not probabilities), it is badly distorted by extreme feature values, and its loss function is the wrong shape for a yes/no target. Logistic regression fixes all three at once.

# 2 Logistic regression

We keep linear regression's engine — a weighted sum of the features — and add one component that converts that sum into a probability. Start from the sum you already know,

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p,$$

where, for our subscriber, $x_1$ might be watch hours, $x_2$ tenure, and so on. This $z$ (the **log-odds**, as we will see) ranges over all real numbers — exactly the problem. We need to squash it into $(0, 1)$.

## 2.1 The sigmoid function

**Definition 2** The **sigmoid** (or **logistic**) function maps any real number to the open interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It is increasing, passes through $\sigma(0) = 0.5$, and saturates toward 1 as $z \to +\infty$ and toward 0 as $z \to -\infty$.

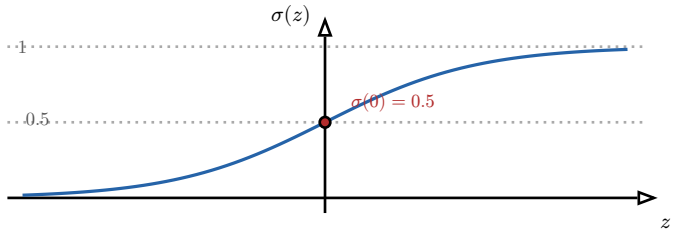


Figure 1: The sigmoid function. It takes the unbounded linear score $z$ and returns a number in $(0, 1)$ we can read as a probability. Large positive $z$ gives near-certainty of the positive class; large negative $z$, near-certainty of the negative class; $z = 0$ is maximal uncertainty at 0.5.

**Definition 3** **Logistic regression** models the probability of the positive class as the sigmoid of the linear score:

$$\hat{p} = P(y = 1 \mid x) = \sigma\big(\beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p\big).$$

The fitted $\hat{p}$ is a probability; the predicted class is obtained by comparing $\hat{p}$ to a **decision threshold** (0.5 by default).

## 2.2 Interpreting the output and the coefficients

The output $\hat{p}$ is read directly: $\hat{p} = 0.87$ means the model assigns this subscriber an 87% chance of churning. That probabilistic reading is one of logistic regression's great virtues — it is **calibrated language** a business can act on.

The coefficients need a little more care than in linear regression, because the sigmoid sits between $z$ and the probability. Rearranging the definition gives the clean interpretation:

$$\underbrace{\frac{\hat{p}}{1-\hat{p}}}_{\text{odds}} = e^{\beta_0+\beta_1 x_1+\cdots}, \quad \text{so} \quad \ln\left(\frac{\hat{p}}{1-\hat{p}}\right) = \beta_0 + \beta_1 x_1 + \ldots$$

The linear score $z$ is the **log-odds**. So each coefficient $\beta_j$ is the change in log-odds per one-unit increase in $x_j$, all else equal; equivalently, $e^{\beta_j}$ multiplies the **odds**.

> **Worked example — reading a churn coefficient.** Suppose the fitted coefficient on "number of support tickets last month" is $\beta_j = 0.5$. Then each additional support ticket multiplies the odds of churn by $e^{0.5} \approx 1.65$ — a 65% increase in the odds, all else equal. A coefficient of 0 means the feature multiplies the odds by $e^0 = 1$, i.e. has no effect. The **sign** is the quick read: positive pushes toward churn, negative toward staying.

**Common pitfall** A logistic coefficient is a change in **log-odds**, not a change in probability. The same one-ticket increase moves the probability a lot near $\hat{p} = 0.5$ (the steep part of the sigmoid) and almost not at all near $\hat{p} = 0.99$ (the flat part). "An extra ticket adds 0.12 to the probability" is wrong — the effect on probability depends on where you start.

### 2.3 How it is fitted

We cannot use the residual sum of squares here — a 0/1 target with the sigmoid makes RSS a bumpy, hard-to-optimise surface. Logistic regression is fitted by minimising a different loss, **log loss** (cross-entropy), which rewards confident correct probabilities and punishes confident wrong ones.

**Definition 4** The **log loss** (binary cross-entropy) for predictions $\hat{p}^{(i)}$ against labels $y^{(i)} \in \{0, 1\}$ is

$$L(\beta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)} \ln \hat{p}^{(i)} + \left(1 - y^{(i)}\right) \ln\left(1 - \hat{p}^{(i)}\right)\right].$$

For a positive example ($y = 1$) only the first term survives, so the loss is $-\ln \hat{p}$: small when $\hat{p}$ is near 1, exploding as $\hat{p} \to 0$.

This loss is convex, so the gradient descent you implemented in DAT32-1 minimises it without local-minimum trouble; in practice scikit-learn's `LogisticRegression` does it for you. The point to carry forward is conceptual: **same linear score, same gradient-descent machinery, a sigmoid on the output and a log loss instead of squared error.**

## 3 The confusion matrix and the core metrics

We have probabilities and, after thresholding, predicted classes. Now: is the classifier any good? "What fraction did it get right?" is the first instinct and the first trap. Every classification metric is built from one small table, so we build it first.

## 3.1 The confusion matrix

**Definition 5** For binary classification with a chosen threshold, the **confusion matrix** cross-tabulates predicted class against actual class into four counts:

- **True Positive** (TP): predicted positive, actually positive.
- **False Positive** (FP): predicted positive, actually negative — a **false alarm**.
- **False Negative** (FN): predicted negative, actually positive — a **miss**.
- **True Negative** (TN): predicted negative, actually negative.

By convention the **positive** class is the event of interest (here, "churn").

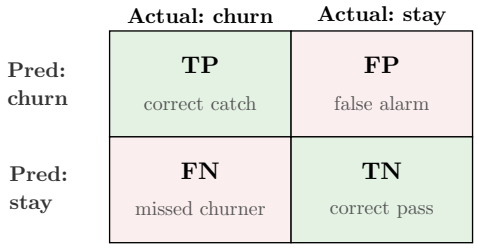


Figure 2: The confusion matrix for churn prediction. The green diagonal is correct decisions; the red off-diagonal is the two kinds of error. A false negative (FN) is a churner you failed to call; a false positive (FP) is a loyal customer you called for nothing. Almost every metric below is a ratio of these four cells.

## 3.2 Accuracy and why it lies

**Definition 6** **Accuracy** is the fraction of correct predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

Accuracy is intuitive and, on its own, often useless — because real classification problems are **imbalanced**.

> **Worked example — the 97%-accurate model that is worthless.** Only 3% of your subscribers churn in a given month. A "model" that predicts **stay** for everyone is 97% accurate — it is right on all 97% who stay. Yet it catches zero churners, so the retention team gets an empty call list. Its high accuracy is an artefact of the class imbalance, not evidence of skill. Accuracy rewards the model for the easy majority and stays silent about the rare class you actually care about.

## 3.3 Precision and recall

The fix is to ask two sharper questions, each about the positive class.

**Definition 7** **Precision** answers "of those we flagged positive, how many truly were?":

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

**Recall** (sensitivity, true-positive rate) answers "of all true positives, how many did we catch?":

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Read them through the costs. **Precision** is high when false alarms are rare — it is the metric to watch when **acting on a positive is expensive**. **Recall** is high when misses are rare — the metric to watch when **missing a positive is expensive**.

**Worked example — which one the business needs.** A **cancer screening** test must not miss a real tumour; a missed case (FN) can be fatal, while a false alarm (FP) costs a follow-up scan. So it is tuned for **high recall**, accepting more false positives. A **spam filter** must not send a real, important email to the junk folder; a false positive (a real email marked spam) is far worse than a false negative (one spam reaching the inbox). So it is tuned for **high precision**. The same model, the same maths — opposite priorities, set by the cost of each error.

**Common pitfall** Precision and recall trade off against each other and you can trivially max out either alone. Flag **everyone** as positive and recall is a perfect 100% (you missed no one) while precision collapses. Flag only your single most certain case and precision may be 100% while recall is near zero. A number quoted without its partner is meaningless — always report precision **and** recall together.

### 3.4 The F1 score

When you need a single number that refuses to be fooled by either extreme, combine the two.

**Definition 8** The **F1 score** is the harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The harmonic mean is close to the **smaller** of the two, so F1 is high only when **both** precision and recall are high. A model that scores 1.0 on one and 0.0 on the other gets F1 $= 0$, not 0.5.

**Worked example — why harmonic, not arithmetic.** The "flag everyone" model has recall 1.0 and precision, say, 0.03. The arithmetic mean would be a misleadingly respectable 0.515; the harmonic mean is $2(0.03 \cdot 1.0)/(0.03 + 1.0) \approx 0.058$ — correctly near zero. F1 punishes imbalance between precision and recall, which is exactly what you want from a single summary number.

F1 is the default single-number metric for imbalanced binary problems, but it bakes in an assumption that precision and recall matter **equally**. When they do not — when a miss costs 150 times a false alarm — F1 is the wrong summary and you should go back to reporting precision and recall separately, or weight them deliberately.

## 4 The decision threshold and the ROC curve

So far we thresholded at 0.5 by reflex. But the threshold is a **free dial**, and turning it is the cheapest, most powerful lever in classification — no retraining required.

## 4.1 Threshold as a business lever

**Definition 9** The **decision threshold** $t$ converts a probability into a class: predict positive when $\hat{p} \geq t$. Lowering $t$ flags more cases — **raising recall, lowering precision**; raising $t$ flags fewer — **raising precision, lowering recall**.

**Worked example — turning the dial for the retention team.** At $t = 0.5$ your churn model flags 1 200 subscribers with 60% precision and 55% recall. The team has budget for only 400 calls, and wants those calls to land on real churners — so raise the threshold to $t = 0.8$: now it flags 350 high-certainty cases at 85% precision (fewer wasted calls) but 30% recall (more churners slip through). If instead a churner is very costly and calls are cheap, lower $t$ to 0.3 to catch nearly everyone at the price of many false alarms. Same model, same coefficients — the threshold alone re-balances the two errors to fit the budget and the costs.

**Common pitfall** The default 0.5 threshold is a convention, not a law, and it is rarely optimal on an imbalanced or cost-asymmetric problem. Choosing the threshold is a **business decision** driven by the relative cost of a false positive versus a false negative — not a setting to leave at its default. And like any tuning choice, pick it on validation data, never on the test set.

## 4.2 The ROC curve

A model that has to commit to one threshold hides how it would behave at every other. The ROC curve shows them all at once, which lets you compare **models** independently of any single threshold.

**Definition 10** The **ROC curve** (Receiver Operating Characteristic) plots the **true-positive rate** (recall, $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$) against the **false-positive rate** ($\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$) as the threshold sweeps from 1 down to 0. Each point is the model at one threshold; the diagonal is a random classifier.

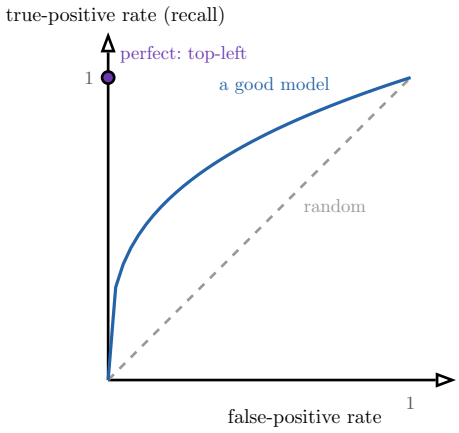


Figure 3: An ROC curve. Sweeping the threshold traces the curve from the top-right (threshold 0: flag everything) to the origin (threshold 1: flag nothing). A model that bows toward the top-left corner — high recall at low false-alarm rate — is strong; the dashed diagonal is a coin flip. The area under the curve summarises the whole picture in one number.

**Definition 11** The **AUC-ROC** (Area Under the ROC Curve) summarises the curve as a single number in $[0, 1]$. It equals the probability that the model scores a random positive example higher than a random negative one. $\text{AUC} = 1$ is perfect ranking, $\text{AUC} = 0.5$ is random, and AUC is **threshold-independent** — it measures how well the model **ranks** cases, separate from where you eventually cut.

**Worked example — AUC compares models, the threshold tunes one.** You compare two churn models and model A has AUC 0.88 versus model B's 0.81 — A ranks churners above stayers more reliably, **at every threshold**, so you pick A. Only then do you choose A's operating threshold from the business costs. AUC is for choosing the model; the threshold is for operating it. Do not confuse the two decisions.

**Common pitfall** AUC can look reassuringly high on a severely imbalanced problem even when precision at your chosen threshold is poor, because the false-positive rate has a huge negative denominator that hides a lot of false alarms. When positives are rare and precision is what matters, complement AUC-ROC with the **precision–recall curve** and report precision and recall at the threshold you will actually deploy.

### 4.3 Communicating results in plain language

A confusion matrix and an ROC curve are inputs to a **decision** made by people who do not know what recall is. Translate. Instead of "recall is 0.55," say "of every 100 customers who will actually leave, today's model catches 55 and misses 45." Instead of "precision is 0.60," say "of every 100 customers we call, 60 really were about to leave and 40 were a wasted call." Then connect the dial to money: "lowering the threshold would catch 20 more leavers but add 300 wasted calls — worth it if a saved customer is worth more than those calls." That last sentence, not the metric, is the deliverable.

## 5 Multi-class classification

Churn is binary, but many problems are not: which of five subscription plans will a customer upgrade to, which of a dozen support categories a ticket belongs to. Logistic regression extends to these in two standard ways.

### 5.1 One-vs-rest

**Definition 12** **One-vs-rest** (OvR, also one-vs-all) trains **one binary classifier per class**, each distinguishing its class from all the others combined. To predict, run all $K$ classifiers and choose the class whose classifier outputs the highest probability.

**Worked example — five plans, five classifiers.** For five plans, OvR trains "Basic vs not-Basic," "Standard vs not-Standard," and so on — five logistic regressions. A new customer is scored by all five; if the Standard classifier says 0.71 and every other says below 0.5, predict Standard. OvR is simple and reuses the binary model you already have, which is why it is scikit-learn's default for multi-class logistic regression.

### 5.2 Softmax (multinomial logistic regression)

**Definition 13** **Softmax regression** (multinomial logistic regression) generalises the sigmoid to $K$ classes directly. It computes a linear score $z_k$ per class and converts the whole vector into probabilities that sum to 1:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

The predicted class is the one with the highest probability.

The practical difference: OvR trains $K$ independent models whose probabilities need not be jointly coherent, while softmax fits one model whose $K$ probabilities are guaranteed to form a proper distribution summing to 1. Softmax is usually the cleaner choice when the classes are mutually exclusive; OvR is a robust, simple default and the natural option when you want one tunable model per class.

### 5.3 Evaluating multi-class models

The confusion matrix generalises to a $K \times K$ grid: rows are predicted classes, columns actual, the diagonal correct. Precision, recall and F1 are computed **per class** (treating that class as positive, the rest as negative) and then **averaged** — and how you average matters.

**Definition 14** **Macro-averaging** computes the metric for each class and takes the plain mean, weighting every class equally. **Micro-averaging** pools all classes' TP, FP, FN into one calculation, weighting every **example** equally (so large classes dominate). **Weighted averaging** averages per-class metrics weighted by class size.

**Common pitfall** On imbalanced multi-class data the averaging choice changes the story. Micro-average is dominated by the big classes and can look excellent while a small but important class is handled terribly; macro-average exposes that by giving the rare class equal say. Choose the average to match what you care about — and always state which one you reported, because "F1 = 0.82" is ambiguous until you do.

## 6 Feature scaling

We now turn from evaluation to the preprocessing judgment the OP closes on, starting with scaling. Logistic regression sums $\beta_j x_j$, and the regularization penalty (next section) sums $\beta_j^2$ or $|\beta_j|$. Both are distorted when features live on wildly different numeric scales.

**Worked example — when scale silently breaks the model.** Your features are **tenure in months** (range 1–60) and **monthly revenue in euros** (range 5–40) and **total watch minutes** (range 0–25 000). Watch minutes dwarf the others numerically. Under L2 regularization, which penalises large coefficients, the model is pushed to keep the watch-minutes coefficient tiny simply because the feature's numbers are huge — not because the feature is unimportant. Distance- and penalty-based methods are silently dominated by whichever feature happens to be measured in big units. Scaling removes this accident.

**Definition 15** **Standardization** rescales a feature to zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma},$$

using the feature's training-set mean $\mu$ and standard deviation $\sigma$. Values become "number of standard deviations from the mean," unbounded but centred.

**Definition 16** **Normalization** (min–max scaling) rescales a feature to a fixed range, usually $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Values are bounded but sensitive to outliers, since a single extreme $x_{\max}$ compresses everyone else.

When does it matter? **Standardization** is the safe default for linear and logistic models, especially with regularization, and is more robust to outliers than min–max. **Normalization** suits features you want bounded (e.g. for certain neural nets or when a feature is already a bounded proportion). And crucially — **when does it not matter?**

**Worked example — when scaling changes nothing.** A plain, **un-regularized** logistic regression is essentially invariant to feature scaling: rescale a feature by a constant and the fitted coefficient simply rescales to compensate, leaving predictions unchanged. Decision trees and random forests are likewise scale-invariant — they split on thresholds, and a monotone rescaling does not change the order. Scaling matters for **penalised** linear/logistic models and distance-based methods; it is wasted effort for trees. Knowing when **not** to scale is as much a part of the judgment as knowing when to.

**Common pitfall** Fit the scaler on the **training data only**, then apply its stored $\mu, \sigma$ (or $x_{\min}, x_{\max}$) to validation and test data. Computing the mean and standard deviation over the whole dataset leaks information from the held-out sets into training and inflates your scores — exactly the leak that scikit-learn pipelines, from DAT32-1, exist to prevent. Put the scaler **inside** the pipeline.

# 7 Feature engineering for linear models

A linear model can only draw straight boundaries in the features you give it. Its power therefore lives largely in **which features you build**. Three constructions do most of the work.

## 7.1 Polynomial features

**Definition 17** **Polynomial features** add powers of existing features $(x, x^2, x^3, \ldots)$ so a linear model can fit curved relationships. The model stays linear **in its coefficients** — it is still $\beta_0 + \beta_1 x + \beta_2 x^2$ — but now bends as a function of $x$.

This is exactly the polynomial regression of DAT32-1, reframed as a **feature construction**: you are not changing the model, you are enriching its inputs. The same overfitting risk applies —

high-degree features fit noise — so they pair naturally with regularization and cross-validated degree selection.

### 7.2 Interaction terms

> **Worked example — when two features only matter together.** Among your subscribers, a high price alone is tolerable and low watch hours alone is tolerable, but a customer paying a **high price** who **barely watches** is a churn risk far beyond what either feature predicts on its own. A plain linear model, adding the two effects separately, cannot express "high price **and** low usage." The **product** of the two features can.

**Definition 18** An **interaction term** is a new feature formed by multiplying two features, $x_i \times x_j$. It lets a linear model capture effects that depend on **combinations** of features — situations where the impact of one feature changes with the value of another.

### 7.3 Encoding categorical variables

Linear models consume numbers, but features like **subscription plan** (Basic / Standard / Premium) or **country** are categories. They must be encoded — and the naive encoding is a trap.

**Definition 19** **One-hot encoding** turns a categorical feature with $K$ levels into $K$ binary (0/1) indicator features, exactly one of which is 1 per row. **Label (ordinal) encoding** maps the levels to integers $0, 1, 2, \ldots$.

**Common pitfall** Do not label-encode an **unordered** category. Mapping Basic→0, Standard→1, Premium→2 tells the model Premium is "three Basics" and that Standard sits exactly between — a false ordering it will dutifully exploit, fitting nonsense. Use one-hot encoding for unordered categories; reserve integer/ordinal encoding for genuinely ordered ones (e.g. small < medium < large). With many levels, one-hot can explode the feature count, which is itself a reason to lean on L1 regularization next.

## 8 Regularization for logistic regression

Regularization carries over from DAT32-1 essentially unchanged — the same L1 and L2 penalties, added now to the log loss instead of the RSS — and the same intuition for which to prefer. It matters more here because feature engineering above can multiply your feature count (polynomials, interactions, one-hot levels) and invite overfitting.

**Definition 20** **Regularized logistic regression** adds a coefficient penalty to the log loss:

$$L_{\text{L2}} = \text{log loss} + \lambda \sum_{j=1}^{p} \beta_j^2, \quad L_{\text{L1}} = \text{log loss} + \lambda \sum_{j=1}^{p} |\beta_j|.$$

As in linear models, **L1 (lasso)** drives some coefficients exactly to zero — **automatic feature selection** — while **L2 (ridge)** shrinks coefficients smoothly toward zero for **stability**, never quite eliminating them.

The choice mirrors DAT32-1. Prefer **L1** when you suspect many of your engineered features are useless and you want a sparse, interpretable model that names the few that matter — invaluable

after a one-hot or polynomial expansion has produced hundreds of candidate features. Prefer **L2** when you believe most features contribute a little and you mainly want to keep coefficients stable, especially when features are correlated (as interaction and polynomial terms inevitably are with their parents).

**Common pitfall** scikit-learn's `LogisticRegression` is regularized **by default** (L2, strength set by `C`, where $\texttt{C} = \frac{1}{\lambda}$ — so **smaller** `C` means **stronger** regularization). This trips up the unwary: the model you get "out of the box" is already penalised. Set `C` deliberately and tune it by cross-validation, as you tuned $\lambda$ for ridge. And — as everywhere — scale features before penalising them, or the penalty falls unevenly.

## 9 When simple models win

You could reach for a gradient-boosted forest or a neural network for churn, and sometimes you should. But this course argues, deliberately, that logistic regression is often the **right** model — not merely the easy one — and naming why is part of the judgment the OP demands.

**Interpretability.** A logistic regression hands you a coefficient per feature: "each support ticket raises the odds of churn by 65%." When the retention team, a regulator, or a sceptical executive asks **why** a customer was flagged, you can answer in a sentence. A deep model's answer is "the weights say so." In regulated domains (credit, insurance, hiring) this is not a nicety — an unexplainable decision can be illegal.

**Data efficiency.** With a few thousand rows, a simple model with few parameters generalises better than a flexible one starved of data. Complex models need large data to earn their flexibility; below that, they overfit and a logistic regression wins outright.

**Inference speed and operational cost.** A logistic regression scores a customer with one dot product and one sigmoid — microseconds, trivial to deploy, easy to monitor, cheap to retrain. A large model can cost orders of magnitude more to serve at scale and is far harder to debug when it drifts.

> **Worked example — the model that ships beats the model that impresses.** A boosted forest scores AUC 0.91 against logistic regression's 0.88. The forest wins on paper. But the logistic model can be explained to the compliance team, retrained nightly on a laptop, and its flagged reasons handed to the retention staff as talking points. On a 3-point AUC gap, the simpler model that the whole organisation can understand, audit and operate is frequently the better **business** decision. "Best model" is not "highest score" — it is best given accuracy, interpretability, data and cost together.

**Common pitfall** The converse error is real too: never reach for complexity **by reflex**. The right order is to make the simple model work, evaluate it honestly with the toolkit above, and adopt a complex model only when the measured gain justifies the cost in interpretability and operations. Complexity is a cost you pay, not a virtue you earn.

## 10 The end-to-end classification pipeline

Everything in this book assembles into one reproducible workflow — the scikit-learn pipeline of DAT32-1, now carrying a classifier and the classification evaluation toolkit. The shape of a complete classification project:

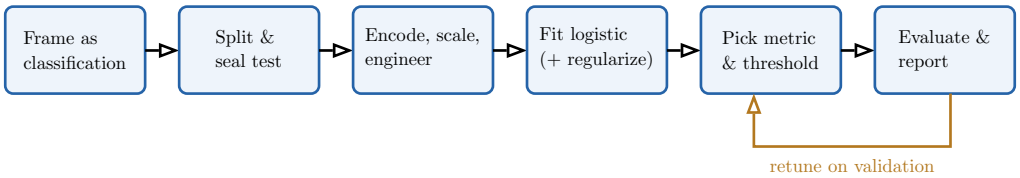


Figure 4: The end-to-end classification pipeline. Preprocessing (encode, scale, engineer) and the model live inside one scikit-learn pipeline so every step is fitted on training data only and re-applied identically under cross-validation and at test time. The metric and threshold are chosen on validation from the business costs; the test set is opened once, at the end, for the report.

The deliverable is not a number but a **business-oriented evaluation report**: which metric you chose and why (in terms of the cost of a false positive versus a false negative), the confusion matrix translated into plain language, the operating threshold and what it buys, and an honest statement of what the model gets wrong. That report — model plus justified evaluation plus clear communication — is the whole course in one artefact.

**The arc of this course.** Classification asks "which?" and "yes/no?" instead of regression's "how much?", and that one change ripples outward: the output becomes a probability (logistic regression and the sigmoid), errors become asymmetric (the confusion matrix, precision, recall, F1), the threshold becomes a business lever (ROC, AUC), and the model extends to many classes (one-vs-rest, softmax). Preprocessing — scaling, feature engineering, encoding — and regularization make the linear model strong, while judgment about interpretability, data and cost decides when simple is best. The hard part was never the model; it was choosing and defending how you measure it.

**Exercises**

**1.** For five business problems, state whether each is a classification task and, if so, name the positive class and which error (false positive or false negative) is more costly. Justify each from the business context.

**2.** Implement logistic regression and apply it to a labelled binary dataset. Interpret three coefficients in terms of odds ($e^{\beta_j}$), and explain why the effect of a feature on the **probability** depends on the starting point.

**3.** Plot the sigmoid function and mark $\sigma(0)$. Explain, using the curve, why a one-unit change in a feature moves the predicted probability more near $\hat{p} = 0.5$ than near $\hat{p} = 0.99$.

**4.** On an imbalanced dataset, build the confusion matrix and compute accuracy, precision, recall and F1. Exhibit a trivial majority-class predictor with high accuracy but zero recall, and explain why accuracy is the wrong metric here.

**5.** For two contrasting problems (one where misses are costly, one where false alarms are costly), select and justify precision, recall or F1, and state which class is positive.

**6.** Sweep the decision threshold from 0 to 1 and plot precision and recall against it. Pick a threshold for a stated budget/cost scenario and justify it. Explain why the threshold must be chosen on validation, not test, data.

**7.** Plot a ROC curve for a fitted model and compute its AUC. Compare two models by AUC, then explain why AUC chooses the model while the threshold operates it.

**8.** Take a confusion matrix and write a two-paragraph plain-language report for a non-technical manager: what the model catches, what it misses, what each error costs, and what moving the threshold would do.

**9.** Train a multi-class classifier with one-vs-rest and with softmax. Compare their predictions, build the $K \times K$ confusion matrix, and report macro- and micro-averaged F1, explaining when the two disagree and why.
**10.** Apply standardization and min–max normalization to features on different scales. Show that an L2-regularized logistic regression's coefficients change with scaling, and that an unregularized one's predictions do not. State when scaling is necessary and when it is irrelevant.
**11.** Engineer features for a linear model: add polynomial terms, an interaction term motivated by the domain, and one-hot encode a categorical variable. Show a case where label-encoding an unordered category produces a worse model, and explain why.
**12.** Apply L1 and L2 regularization to a logistic regression with many engineered features. Report which coefficients L1 drives to zero, tune `C` by cross-validation, and give a situation where each penalty is the better choice.
**13.** Build a complete scikit-learn pipeline (encode + scale + engineer + regularized logistic regression), evaluate it under cross-validation with a justified metric and threshold, and produce a one-page business-oriented evaluation report.